Space-Guard

A System That Watches Satellites and Prevents Collisions in Space

Smart India Hackathon 2026 — Software Edition · Problem ID: 17

Team Space-Guard

August 18, 2026

Abstract

This document is the full presentation script and slide content guide for the Space-Guard project at SIH 2026. Space-Guard is a software system that monitors satellites orbiting Earth, detects when two satellites are getting dangerously close to each other, and automatically calculates the safest, most fuel-efficient way to move one of them out of the way — before a collision can happen.

Presenter Note

How to use this document: Read the slides out loud to the judges during your presentation. The orange boxes (like this one) are your personal notes — they tell you what to say and why. The judges will not see these notes; they are only for you.

Contents

1 Slide 1: Title and Team Introduction **2**

2 Slide 2: The Problem — Why This Matters **3**

3 Slide 3: Our Solution — How Space-Guard Fixes This **5**

4 Slide 4: How We Built It — Technical Implementation **6**
- 4.1 Part 1: Physics and Math . . . 6
- 4.2 Part 2: Software Architecture . . . 7
- 4.3 Part 3: Where Do We Get the Satellite Data? . . . 7

5 Slide 5: Live Demo Walkthrough **8**

6 Slide 6: Who Would Use This? (Target Audience) **9**

7 Slide 7: Roadmap — What We Plan Next **10**

1 Slide 1: Title and Team Introduction

Key Information

Project: Space-Guard — A Satellite Collision Prevention System
Theme: Space Technology / Smart India Hackathon 2026
Tagline: *"We watch the skies so astronauts and satellites stay safe."*

Presenter Note

Say something like this: "Hi, we are Team Space-Guard. Our project is a software system that tracks satellites in orbit and automatically detects when two of them are on a collision course — and then tells operators exactly how to steer one of them out of the way, using as little fuel as possible. Think of it like Google Maps, but for satellites, with a built-in emergency alert system."

What Each Team Member Built

- **Team Lead and Software Developer**:
  - Built the website (frontend) and the server that runs behind the scenes (backend).
  - Connected the system to live satellite data from public space agency databases.
  - Made everything work together end-to-end.
- **Physics and Math Lead**:
  - Wrote the code that calculates where each satellite is in space, right now, using standard physics equations.
  - Designed the two-step system that quickly finds which satellites are actually at risk of hitting each other (instead of checking every pair, which would be too slow).
  - Calculated the exact probability (in numbers) that two satellites will crash.
- **Collision Avoidance Lead**:
  - Wrote the code that figures out the best way to steer a satellite away from danger.
  - Optimized the engine burn to use the least possible fuel.
  - Proved that acting early saves 14 times more fuel than acting at the last minute.
- **3D Visualization and Data Engineer**:
  - Built the interactive 3D Earth globe in the browser.
  - Made the 2009 collision replay animation showing the crash and debris.
  - Ensured the 3D view runs smoothly in any web browser, without any special software.

Presenter Note

Point to each team member as you say their name and contribution. Keep it short — 1 sentence per person. If a judge asks about a specific part, the relevant person should answer.

2 Slide 2: The Problem — Why This Matters

The Situation in Space Today

- There are currently over **27,000 man-made objects** orbiting Earth, including active satellites, dead satellites, and old rocket parts.
- These objects travel at speeds of **27,000 km/h** — about 40 times faster than a bullet.
- In the last five years, companies like SpaceX (Starlink) have launched thousands more satellites, making space even more crowded.
- When two objects collide, they shatter into thousands of tiny pieces that then threaten even more satellites — a chain reaction called the **Kessler Effect**.

Presenter Note

Use this analogy: "Imagine a busy highway where all the cars are moving at 40 times the speed of a bullet, and nobody has a GPS, a horn, or brakes. That is low Earth orbit today. One crash can cause a chain reaction that makes all of space unusable for decades." This makes it real for the judges.

Real Proof: The 2009 Iridium–Cosmos Collision

Real-World Case Study

On **10 February 2009 at 16:56 UTC**, at **789 km above Siberia**:

- An active American phone satellite called **Iridium 33** (560 kg, roughly the size of a small car) hit a dead Russian military satellite called **Cosmos 2251** (900 kg).
- They were moving towards each other at **14.12 km/s** (over 50,000 km/h) — a near head-on crash.
- Both satellites were completely destroyed in milliseconds.
- The crash created over **2,000 trackable pieces of debris** and thousands of smaller, untrackable fragments that are still flying around in orbit today, threatening the International Space Station and other satellites.
- **This was entirely preventable** — nobody detected the risk early enough to do anything about it.

Presenter Note

This is the most important part of the presentation — make the judges feel the impact of this event. Say: "This is not a hypothetical. This actually happened. Two real satellites crashed. Both were completely destroyed. And the wreckage is still up there today, threatening the ISS astronauts every single day. Our project exists to make sure this can never happen again." You can also say: "At the time, there was no system fast enough or smart enough to warn operators in time. We built Space-Guard to be that system."

Why Existing Systems Are Not Enough

1. **Too slow**: Checking all 27,000 satellites against each other manually takes hours. By the time the calculation is done, there is no time to act.

2. **Too many false alarms**: Simple distance-based systems trigger thousands of warnings per day. Operators get flooded and start ignoring them.

3. **No automated escape plan**: Even when a risk is detected, figuring out the best engine burn to dodge it still requires hours of manual work by engineers.

Presenter Note

Keep this short. You just need to plant the idea that current tools are slow, noisy, and manual. Space-Guard solves all three. You will explain how in the next slide.

3 Slide 3: Our Solution — How Space-Guard Fixes This

Our Solution

Space-Guard is a fully automatic system that:

1. Continuously watches the position of all tracked satellites in real time.
2. Instantly detects which satellites are getting dangerously close to each other.
3. Calculates the exact probability of a crash (in numbers).
4. Automatically designs the safest, most fuel-efficient escape maneuver.

All of this happens in **under 5 seconds** — fast enough to act.

Presenter Note

Say: "Our solution works in three steps. Step one: we pull live satellite positions from public space agency databases. Step two: we run a fast two-stage filter to find which satellites are actually at risk — this cuts down 90% of non-threatening pairs instantly. Step three: for the risky pairs, we calculate the exact crash probability and generate the optimal escape plan automatically. The whole process takes less than 5 seconds."

How We Are Better Than Existing Tools

What It Does	Old / Traditional Tools	Space-Guard
Speed	Minutes to hours	**Under 5 seconds**
Risk Calculation	Simple distance only	**Real probability number (e.g., 1 in 5,000)**
AI / Smart Filtering	Usually none	**Pre-screens pairs to skip obvious non-threats**
Escape Plan	Manual, by engineers	**Fully automatic, optimized for minimum fuel**
Fuel Savings	Last-minute emergency burns	**14× more efficient when done 24h early**
Real-World Testing	Synthetic fake data	**Tested on the actual 2009 Iridium-Cosmos crash**

Table 1: Space-Guard vs. Legacy Conjunction Tools.

Presenter Note

Walk through this table slowly. Point to each row. Emphasize the last row — the fact that we tested on the real 2009 crash data and our system correctly identifies it as a critical risk. That is our "proof that it works." Judges love validation on real historical data.

4 Slide 4: How We Built It — Technical Implementation

Presenter Note

This is the technical slide. You do not need to explain every formula. Just explain the *idea* behind each part. Say: "I will explain how each part of the system works. I will keep it as simple as possible." If the judges want more depth, they will ask.

4.1 Part 1: Physics and Math

A. Where Are the Satellites Right Now?

Every satellite's official position data is published as a pair of text lines called a **Two-Line Element set (TLE)**. These lines contain coded numbers describing the orbit. We use a standard physics model called **SGP4** to decode these numbers and calculate exactly where in space the satellite is right now (as X, Y, Z coordinates relative to the center of Earth), and how fast it is moving.

Presenter Note

Say: "Think of a TLE like a satellite's address. We take that address and use physics equations to calculate the satellite's exact GPS-style position at any moment in time."

B. How Do We Find Which Satellites Are at Risk?

Checking every possible pair of 27,000 satellites would mean over 360 million comparisons. That is too slow. We use a **two-step filter**:

- **Step 1 (Fast Height Check)**: We instantly discard all pairs where the two satellites fly at completely different heights. If one satellite is at 400 km and another is at 800 km, they can never collide. This eliminates 90%+ of pairs in milliseconds.
- **Step 2 (Precise Close-Pass Check)**: For the remaining pairs, we calculate the exact closest distance the two satellites will ever reach, and at what time. This pinpoints the exact "Time of Closest Approach."

Presenter Note

Good analogy: "Imagine you want to find which cars on a highway might crash. First, you ignore all cars going in completely different directions. Then, for the remaining ones, you track their exact paths to find when they get closest. We do the same thing, but for satellites."

C. What Is the Actual Chance of a Crash?

Once we find a close pass, we calculate the exact mathematical probability of a collision. We consider the uncertainty in our position data (satellite locations are not perfectly known — there is some error) and the physical size of each satellite. The result is a number, like "1 in 50,000" or "1 in 5,000." We flag anything above "1 in 10,000" as dangerous.

The formula used is an established aerospace standard:

$$P_c \approx \frac{\mathrm{HBR}^2}{2\sigma^2} \exp\left(-\frac{d_{\mathrm{miss}}^2}{2\sigma^2}\right) \tag{1}$$

where HBR is the combined size of both satellites (about 10 metres), d_{miss} is how close they get, and σ is the uncertainty in our position measurements.

Presenter Note

You do not need to explain the formula. Just say: "We calculate a precise crash probability, like a 1-in-5,000 chance of collision. This is a much more useful number than just saying the two satellites are 5 km apart, which doesn't tell you if that is dangerous or not."

D. How Do We Steer the Satellite to Safety?

We use a physics model (called the Clohessy-Wiltshire equations) that describes how two satellites move relative to each other in orbit. We then use a mathematical optimization technique (SVD — Singular Value Decomposition) to find the single best direction and smallest possible engine push that will move the satellite far enough away to be safe.

The 14× Fuel Savings Rule: If you push the satellite 24 hours before the predicted collision, the satellite drifts along its orbit path and ends up 14 times further away than if you had used the same engine push just 1 hour before the collision. Acting early is dramatically more efficient.

Presenter Note

Great analogy for the 14× rule: "It is like steering a ship. If you turn the wheel slightly 10 km before a reef, you avoid it easily. If you wait until you are 100 metres away, you have to turn hard and fast, and you might still crash. Same idea with satellites — early action means less fuel and more safety."

4.2 Part 2: Software Architecture

- **Backend (Python/FastAPI)**: The server that does all the calculations. Runs on Python. Handles live satellite data, collision screening, and escape plan generation.
- **Frontend (React / Web Browser)**: The interactive website/dashboard that shows everything visually — the 3D globe, the satellite table, the collision replay, the escape plan sliders.
- **Smart Pre-Filter (Machine Learning)**: A simple pre-trained model that quickly scores satellite pairs before doing the full calculation, skipping obvious non-threats even faster.

4.3 Part 3: Where Do We Get the Satellite Data?

- We pull live data from **CelesTrak** — a free public database maintained by aerospace engineers that publishes the position data of all tracked satellites, updated multiple times per day.
- We cache this data locally so we do not hit rate limits and can respond instantly.

Presenter Note

Say: "All the satellite data we use is publicly available. Space agencies share this data openly so that everyone can help keep space safe. We just built the smart software on top of it."

5 Slide 5: Live Demo Walkthrough

Presenter Note

This is where you open the browser and show the working prototype. Open `http://localhost:5173/demo` before the presentation. Have the backend running on port 8000. Walk through each of the 4 steps shown on screen.

The interactive demo is shown at `http://localhost:5173/demo` and has four steps:

1. **Step 1 — Fetch Live Satellites**:
   - Show the table of real satellites with their current positions in space, speeds, and heights above Earth.
   - Hit "Refresh" to show it pulling fresh live data in real time.
   - *Say: "This is live data. These are real satellites flying overhead right now."*
2. **Step 2 — Show on the 3D Globe**:
   - Show the spinning 3D Earth with satellite dots at their correct positions.
   - Point out satellites at different heights and angles (polar vs equatorial orbits).
   - *Say: "Each dot is a real satellite. Our system watches all of them simultaneously."*
3. **Step 3 — Replay the 2009 Crash**:
   - Show the animated collision between Iridium 33 and Cosmos 2251 on the globe.
   - Point to the stats: 14.12 km/s, 3-metre miss distance, 2,000+ debris pieces.
   - *Say: "This is the actual 2009 collision, replayed using the real satellite data from that day. Our system correctly identified this as a critical risk."*
4. **Step 4 — Show the Escape Plan**:
   - Show the avoidance simulation with the interactive sliders.
   - Drag the "How early we fire" slider from 1 hour to 24 hours and show the gap increasing dramatically.
   - *Say: "With a tiny engine push — less than walking speed — applied one day early, the satellites miss each other by over 4 kilometres. Crash risk drops to essentially zero. This is the escape plan our system would have given in 2009."*

Presenter Note

Practice this demo at least 3 times before the presentation. Know exactly what to click. If the live API is down, the system falls back to offline demo data automatically — it will still look the same. Do not panic if the internet is slow.

6 Slide 6: Who Would Use This? (Target Audience)

- **ISRO (Indian Space Research Organisation)**:
  - ISRO manages Indian satellites like Cartosat (mapping), GSAT (communication), and the Chandrayaan moon missions.
  - Space-Guard can be integrated into ISRO's existing space monitoring center (IS4OM at ISTRAC, Bangalore) to protect India's space assets.
- **Private Satellite Companies (e.g., SpaceX Starlink, OneWeb, Pixxel)**:
  - These companies have hundreds to thousands of satellites. Manual monitoring is impossible.
  - Space-Guard provides automated screening, reducing the burden on operators.
- **Defence and Military Space Programs**:
  - Tracking and protecting military satellites from non-cooperative objects and debris.
- **Space Insurance Companies**:
  - Space-Guard provides accurate, evidence-based collision risk numbers that insurance underwriters can use to price policies fairly.

Presenter Note

Focus on ISRO when talking to SIH judges — it is the most relevant and impressive connection. Say: "ISRO has its own space safety center called IS4OM. Our system is designed to be exactly the kind of tool they need to protect India's growing fleet of satellites." This shows you did your research.

7 Slide 7: Roadmap — What We Plan Next

Presenter Note

This slide shows judges that you have a vision beyond the hackathon. Keep it brief. Say: "We have four phases planned. We have already completed Phase 1 — the working prototype you just saw."

- **Phase 1 (Done — SIH Prototype)**:
  - Live satellite position tracking from CelesTrak public database.
  - Two-step collision screening: fast height filter, then precise closest-pass calculation.
  - Exact crash probability calculation (Foster/Alfano analytic method).
  - **ML surrogate model trained on synthetic data** — a Random Forest model trained on features like miss distance, relative speed, and satellite size to quickly pre-rank dangerous pairs before running the expensive full calculation. This means the expensive physics runs only on the top candidates, not every pair.
  - Automatic escape plan generation using Clohessy-Wiltshire orbital mechanics.
  - Interactive 3D globe visualization in the browser.
  - Validated end-to-end on the real 2009 Iridium-Cosmos crash data — the system independently flags it as Critical 48 hours before impact.
- **Phase 2 (Next 6 Months)**: Scale to the full 27,000+ satellite catalog using GPU-accelerated processing. Integrate official position uncertainty data (CDMs from Space-Track) to replace the assumed uncertainty with real measured values, making crash probability numbers even more accurate.
- **Phase 3 (1 Year)**: Handle entire commercial constellations (e.g., all of Starlink at once). Coordinate maneuvers so multiple satellites can dodge simultaneously without accidentally getting in each other's way. Support satellites with continuous low-thrust electric engines.
- **Phase 4 (2 Years)**: Full integration with space agency mission control standards. Direct telemetry handoff to operators. Real-time 24/7 monitoring service.

Presenter Note

If a judge asks "Is this production-ready?" say: "Not yet — this is a prototype. But the core physics, math, and software are all working correctly on real data. Phase 2 and beyond would bring this to operational readiness with proper infrastructure and funding."